

Explainable Medical Diagnosis Recommender using NARS

Justin Clark and Matthew Setiadi

Dr. Pei Wang

CIS 5590: Artificial General Intelligence

Temple University, Spring 2026

May 3, 2026

1. Introduction

This project builds two complementary medical diagnosis systems on top of Non-Axiomatic Logic (NAL) and uses them to study how belief revision, evidential truth-values, and syllogistic inference behave when the evidence is incomplete or contradictory. Medicine is a domain where clean, closed-world logic does poorly: patients present with overlapping symptoms, tests arrive late, and the same symptom can point to several diseases at once. This makes it a natural testbed for NARS, which is explicitly designed around the Assumption of Insufficient Knowledge and Resources (AIKR).

The project has two parts. The first part (Setiadi) implements Mini-NARS from scratch following Chapter 4 of *Designing a Mind* and applies it to a real patient dataset from Kaggle with 348 records across 116 diseases. The second part (Clark) builds a deterministic NAL diagnosis engine with hand-crafted clinical rules for four respiratory conditions and validates it against the PyNARS reference implementation. Both parts demonstrate the same core NAL properties: deduction produces strong conclusions, revision pools evidence and increases confidence, and contradictory evidence revises beliefs without erasing prior knowledge. The convergence from two independent approaches strengthens the findings.

2. Background: NARS, NAL, and AIKR

NARS is a general-purpose reasoning system that represents beliefs in Narsese and attaches to each statement a truth-value $\langle f, c \rangle$, where f is the frequency (what fraction of the available evidence supports the statement) and c is the confidence (how much evidence the system has). Unlike classical probabilistic systems, NAL does not require that probabilities are well-defined or that the agent has complete information. Every belief is explicitly evidence-based and can be revised when new evidence arrives.

Two NAL operations were central to this project:

Deduction: given an implication rule and an observation, derive a conclusion. In NAL-1, deduction computes $f = f_1 * f_2$ and $c = f_1 * f_2 * c_1 * c_2$. This is strong inference because confidence can remain high. With premises at %1.0;0.9%, deduction produces confidence 0.81.

Revision: when two independent derivations arrive at the same statement, combine them using the NAL revision formula: convert $c/(1-c)$ into an amount of evidence, add the evidence, and convert back. Revision always increases confidence because the total evidence base grows. Two sources with confidence 0.9 and 0.8 merge to confidence 0.93.

The system also uses abduction (reasoning from shared predicates, weak) and induction (reasoning from shared subjects, weak). Both produce confidence below 0.5 with standard inputs, which is the formal expression of the fact that shared symptoms are suggestive but not conclusive evidence.

3. Part One: Mini-NARS Implementation (Setiadi)

3.1 Implementation from Chapter 4

The first contribution is a complete Python implementation of Mini-NARS as specified in Chapter 4 of *Designing a Mind*. This is not a wrapper around PyNARS; it is a ground-up implementation of the inference engine, memory management, and control mechanism. The implementation consists of six core modules:

truth_functions.py implements all NAL truth-value functions using the extended Boolean operators: $\text{and}(x,y) = x*y$, $\text{or}(x,y) = 1-(1-x)(1-y)$, $\text{not}(x) = 1-x$. This includes deduction, abduction, induction, exemplification, revision, and the expectation function $e = c(f-0.5)+0.5$.

bag.py implements the Bag data structure from Section 3.2: a probabilistic priority queue integrated with a hashtable. The Bag uses a distributor array of size $L*(L+1)/2$ where level j appears $j+1$ times, giving higher-priority items proportionally more selection chances. Overflow cascades items downward through bucket levels, with absolute forgetting when the lowest bucket overflows. This is the AIKR mechanism: the system has a hard memory limit and must choose what to keep.

data_structures.py defines Statement (two terms connected by inheritance), TruthValue (frequency, confidence, evidential base), Task (judgment or question to process), Belief (stored knowledge with multiple truth-values sorted by confidence), and Concept (a term plus its task bag and belief bag). The evidential base tracking prevents circular reasoning during revision.

budget_functions.py controls resource allocation using extended Boolean operators. The revision budget rewards novelty. The syllogistic budget rewards strong conclusions. The activate function increases concept priority when new tasks arrive.

mini_nars.py is the core engine containing the Narsese parser, the memory and buffer management, the working cycle, and the complete inference engine with local rules (revision, choice), forward rules (deduction, abduction, induction, exemplification by figure detection), and backward rules (question derivation). Each working cycle selects one concept, fires one task-belief pair, and produces derived tasks.

main.py provides an interactive REPL, a demo test suite with 8 experiments, and file loading for experience files.

3.2 Verification

An automated test suite (test_mini_nars.py) runs 86 assertions across truth-value functions, data structures, Bag behavior, system-level inference, the parser, and a medical domain test. All 86 pass. Key verified results:

Rule	Input	Output	Expected
Deduction	bird->animal, robin->bird	robin->animal %1.00;0.81%	$c = 0.9 * 0.9 = 0.81$
Abduction	bird->animal, fish->animal	fish->bird %1.00;0.45%	$c < 0.5$ (weak)
Induction	water->liquid, water->transparent	transparent->liquid %1.00;0.45%	$c < 0.5$ (weak)
Revision	robin->bird %0.7;0.9% + %0.9;0.8%	robin->bird %0.76;0.93%	c increases
Negative	penguin->bird, penguin->flyer %0.0;0.9%	bird->flyer %0.00;0.45%	$f=0$ preserved

A figure-4 bug was identified and fixed during development. The original code applied exemplification (weak) where deduction (strong) should have been used for the $S \rightarrow M + M \rightarrow P$ premise pair. After the fix, robin->bird + bird->animal correctly produces robin->animal at confidence 0.81 instead of 0.45.

3.3 Medical Application with Kaggle Data

The Kaggle Disease Symptoms and Patient Profile Dataset (348 records, 116 diseases) was loaded and four target diseases with substantial symptom overlap were selected: Influenza, Common Cold, Pneumonia, and Bronchitis. Symptom frequencies were computed directly from the data:

Symptom	Flu	Cold	Pneumonia	Bronchitis
Fever	0.75	0.67	0.88	0.75
Cough	0.75	0.50	0.88	0.75
Fatigue	0.75	0.67	0.88	0.75
Breathing Diff.	0.62	0.33	1.00	1.00

These frequencies became the truth-value frequencies in Narsese statements. Confidence was set to 0.85 for all rules, reflecting the limited sample size (6 to 8 patients per disease). The statements were fed into Mini-NARS and inference cycles were run to derive patient diagnoses.

3.4 Belief Revision Experiment (Setiadi)

Patient symptoms were fed one at a time while tracking how diagnosis expectation changed after each piece of evidence:

Step	New Evidence	Flu	Cold	Pneumonia	Bronchitis
1	Fever	0.64	0.57	0.74	0.64
2	+ Cough	0.71	0.57	0.82	0.71
3	+ Fatigue	0.75	0.65	0.82	0.71
4	+ Breathing Difficulty	0.75	0.36	0.93	0.93
5	Negative Flu Test	0.45	0.65	0.93	0.93

At Step 4, breathing difficulty acted as the decisive symptom. Pneumonia and Bronchitis jumped to 0.93 while Common Cold dropped to 0.36. This reflects the dataset: breathing difficulty appears in 100% of Pneumonia and Bronchitis cases but only 33% of Cold cases. At Step 5, the negative flu test reduced Influenza from 0.75 to 0.45 without erasing the prior evidence from fever, cough, and fatigue. The system revised the belief rather than replacing it.

The system also generated an explanation: *"Pneumonia is the suggested diagnosis because: breathing difficulty is a strong indicator (freq=1.00, conf=0.85), fever is a strong indicator (freq=0.88, conf=0.85), cough is a strong indicator (freq=0.88, conf=0.85)."*

4. Part Two: Deterministic NAL Diagnosis Engine (Clark)

4.1 Design and Scope

The second implementation built a deterministic NAL diagnosis engine using the same truth-value functions but with a different architecture. Rather than implementing the full NARS working cycle with bags and priority-based selection, this engine applies deduction rules directly and deterministically for each piece of evidence, then uses NAL revision to combine all derivations for each disease.

The scope was intentionally narrow: four respiratory/throat conditions (Influenza, Common Cold, COVID-19, Strep Throat) with roughly 40 hand-crafted symptom-disease rules. The values were chosen as rough clinical priors to exercise NAL's belief revision machinery with genuinely overlapping symptoms and distinguishing evidence like loss of smell for COVID-19 and swollen lymph nodes for Strep.

4.2 Knowledge Representation

Every symptom-disease edge was encoded as a universally quantified implication:

```
<<$x --> fever> ==> <$x --> flu>>. %0.80;0.80%
<<$x --> loss_of_smell> ==> <$x --> covid>>. %0.90;0.92%
<<$x --> sore_throat> ==> <$x --> strep>>. %0.90;0.85%
```

A significant engineering challenge was handling negative evidence. A naive encoding of a negative lab as frequency 0 produces a conclusion with confidence 0 under standard deduction (because $c = f_1 * f_2 * c_1 * c_2$ and $f_1=0$ collapses everything). The solution was a *deduction_via_negation* function that recognizes when a rule encodes 'evidence of absence' (low f , high c) and preserves the rule's confidence through the derivation. This is algebraically equivalent to applying standard deduction on the NAL-negated form of the rule and converting back.

4.3 Belief Revision Experiment (Clark)

A scripted experiment interleaved supportive and contradictory evidence:

Step 1: Fever observed. Top: Influenza ($\text{exp}=0.67$). Step 2: Sore throat. Ranking flipped to Strep Throat ($\text{exp}=0.76$). Step 3: Negative rapid strep test. Ranking flipped back to Influenza; Strep's frequency dropped from 0.84 to 0.32 while its confidence increased to 0.91. Step 4: Body aches. Influenza confirmed at $\text{exp}=0.73$.

The critical observation is at Step 3: the system did not simply veto Strep. It combined the strong sore-throat derivation for Strep with the strong 'probably not Strep' derivation from the negative test via

NAL revision. The result was a low-frequency, high-confidence belief ($f=0.32$, $c=0.91$). In NAL, you can simultaneously be very confident and say 'probably not.' This is distinctively NAL behavior.

4.4 Cohort Results

An eight-patient cohort was constructed with both clear and ambiguous presentations. The system achieved 100% accuracy (8/8) on top-ranked diagnosis matching ground truth, including three cases specifically designed to require belief revision for the correct answer to emerge. Cross-checking against the PyNARS Reasoner on patient_alpha showed flu frequency 0.83/confidence 0.76 (PyNARS) versus 0.76/0.85 (deterministic engine), with the same top ranking.

4.5 Explanation Layer

The explanation layer traced each diagnosis back to its supporting evidence. For each disease belief, the system recorded which evidence items contributed, which rules fired, and what truth-values were derived. The explainer produced three outputs per patient: a trajectory showing how each evidence item shifted the ranking, a final ranked list of diagnoses, and a plain-English rationale identifying supporting and contrary evidence with qualitative descriptors (strongly, commonly, weakly, rarely) derived from the rule frequencies.

5. Combined Findings

5.1 Convergence Across Implementations

Both implementations independently confirmed the same NAL properties:

Deduction is strong inference. Setiadi's system produces confidence 0.81 for a single deductive hop ($0.9 * 0.9$). Clark's system shows confidence growing to 0.85 to 0.94 as more rules fire and revise together. In both cases, deduction maintains meaningful confidence.

Abduction and induction are weak inference. Setiadi's system shows both producing confidence below 0.5 ($c=0.45$ with standard inputs). Clark's system does not use abduction/induction explicitly but observes the same phenomenon through low-frequency rules that contribute weak evidence.

Revision pools evidence and increases confidence. Setiadi's system merges %0.7;0.9% and %0.9;0.8% to %0.76;0.93%. Clark's system shows revision accumulating across 5+ evidence items to reach confidence 0.85 to 0.94. Both demonstrate that revision is monotonically confidence-increasing.

Contradictory evidence revises rather than replaces. Setiadi's negative flu test reduces Influenza from 0.75 to 0.45 without erasing fever/cough/fatigue evidence. Clark's negative strep test drops Strep from

0.76 to 0.34 expectation while increasing its confidence to 0.91. Neither system treats contradictory evidence as a veto.

5.2 Domain-Specific Observations

Breathing difficulty is the decisive symptom (Setiadi). In the Kaggle dataset, Pneumonia and Bronchitis both show breathing difficulty in 100% of cases versus 33% for Cold. This single symptom shifts expectation from 0.82 to 0.93 for Pneumonia while dropping Cold from 0.65 to 0.36.

Loss of smell is the decisive symptom (Clark). In the clinical rules, loss of smell has frequency 0.90 and confidence 0.92 for COVID-19, making it the strongest single distinguishing feature. Patient_zeta's ranking flips from Influenza to COVID-19 the moment loss of smell is observed.

Some diseases cannot be distinguished by symptoms alone. Setiadi's system cannot separate Pneumonia from Bronchitis (identical symptom profiles in the dataset). Clark's system cannot separate Flu from COVID-19 without lab tests. Both observations are clinically realistic and demonstrate appropriate uncertainty in the truth-values.

5.3 Differences Between Approaches

Setiadi's implementation uses the full NARS architecture with Bags, probabilistic selection, and the working cycle. This means inference is non-deterministic in timing: the system may need variable numbers of cycles to produce a result depending on which concepts and beliefs are selected. Clark's implementation applies rules deterministically, producing identical results every run.

Setiadi's knowledge base uses real statistical frequencies computed from data. Clark's uses hand-crafted clinical priors. The data-driven approach has the advantage of objectivity but is limited by the small sample size (6 to 8 patients per disease). The hand-crafted approach allows finer control over edge cases but requires domain expertise.

6. Discussion and Limitations

The knowledge bases are small. Four diseases with 16 to 40 rules is a teaching demonstration, not a clinical system. Real diagnostic support would require thousands of rules, hierarchical disease categories, temporal reasoning for symptom onset patterns, and integration with population base rates.

Setiadi's system implements NAL-1 only. Compound terms (NAL-3), temporal reasoning (NAL-7), and goals (NAL-8) are not supported. This means negation must be handled as a workaround (encoding negative evidence as low-frequency positive statements) rather than as a first-class operation.

Clark's `deduction_via_negation` function is a design choice, not a textbook primitive. It falls out of NAL negation algebraically, but applying it automatically based on rule frequency is a heuristic. A stricter implementation would encode the negated implications explicitly.

The PyNARS Reasoner's bag-based scheduler often failed to fire all relevant derivation chains within a bounded number of cycles. This is a known issue with attention-based control in crowded rule bases. Clark's solution was to implement the truth functions directly. Setiadi's Mini-NARS implementation uses a simpler concept set and does not encounter this problem as severely.

The explanations are only as honest as the rule confidences. Both systems confidently state their reasoning, but if the underlying frequencies or clinical priors are wrong, the reasoning is confidently wrong. There is no calibration mechanism.

7. Conclusion

This project built two independent medical diagnosis systems grounded in Non-Axiomatic Logic and used them to study belief revision under incomplete and contradictory evidence. The first system (Setiadi) implements Mini-NARS from scratch with the full Bag-based architecture and applies it to real patient data. The second system (Clark) implements a deterministic NAL engine with clinical rules and validates it against PyNARS.

Both systems demonstrate the properties that motivated using NAL in the first place: beliefs move coherently as evidence arrives, confidence builds through revision, weak inference correctly expresses uncertainty, and contradictory evidence participates in the arithmetic rather than acting as a veto. The convergence of results from two independent implementations, using different architectures, different knowledge sources, and different disease sets, confirms that these are properties of the logic itself, not artifacts of a particular implementation.

The central lesson from this project is that AIKR is not a slogan. It shows up as concrete arithmetic in the truth-value functions. Getting that arithmetic right, especially for evidence of absence and for balancing strong and weak inference, is where most of the engineering work lives.

8. References

Wang, P. (2026). *Designing a Mind: The Implementations of NARS*. Draft.

Wang, P. (2025). *Non-Axiomatic Logic: A Model of Intelligent Reasoning*. 2nd ed. World Scientific.

OpenNARS-4 / PyNARS: <https://github.com/opennars/OpenNARS-4/tree/dev/pynars>

Kaggle Disease Symptoms and Patient Profile Dataset:

<https://www.kaggle.com/datasets/uom190346a/disease-symptoms-and-patient-profile-dataset>